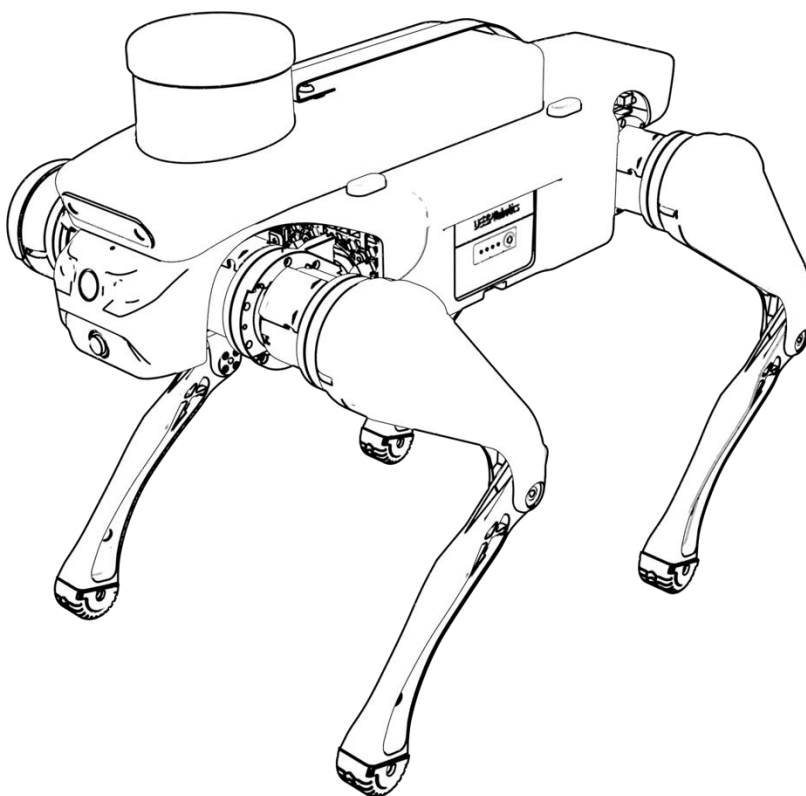


Jueying Lite3

Perception Development Manual(beta)

V2.1.1-0 2024.7.26



Content

1 Perception System	5
2 Preparatory Work	6
2.1 Remote Desktop	6
2.1.1 Connect	6
2.1.2 Troubleshooting	7
2.2 Connect Perception Host Via HDMI	7
2.2.1 Set To Enter The GUI On Startup	8
2.2.2 Restore Entering tty3 On Startup	9
3 Depth Camera	11
3.1 Camera Driver	11
3.2 Camera Test	12
3.3 Realsense Camera Development	13
4 Message Transformer	14
4.1 Introduction	14
4.2 Usage	15
4.3 Development	17
4.3.1 Package Structure	17
5 People Tracking	19
5.1 Introduction	19
5.2 Usage	20
5.3 Development	21

5.3.1 Package Structure	21
6 LiDAR-based SLAM and Navigation	23
6.1 Mapping(for v3.1.04 and later)	23
6.1.1 Introduction.....	23
6.1.2 Usage.....	24
6.2 Mapping(for earlier than v3.1.04)	29
6.2.1 Introduction.....	29
6.2.2 Package Structure.....	29
6.2.3 Usage.....	30
6.3 Localization & Navigation	35
6.3.1 Usage of Point-to-point Navigation.....	35
6.3.2 Usage of Multi-point Navigation	37
6.4 Development	39
6.4.1 Package Structure.....	39
6.4.2 Parameters of Mapping	39
6.4.3 Parameters of Navigation and Obstacle Avoidance.....	40

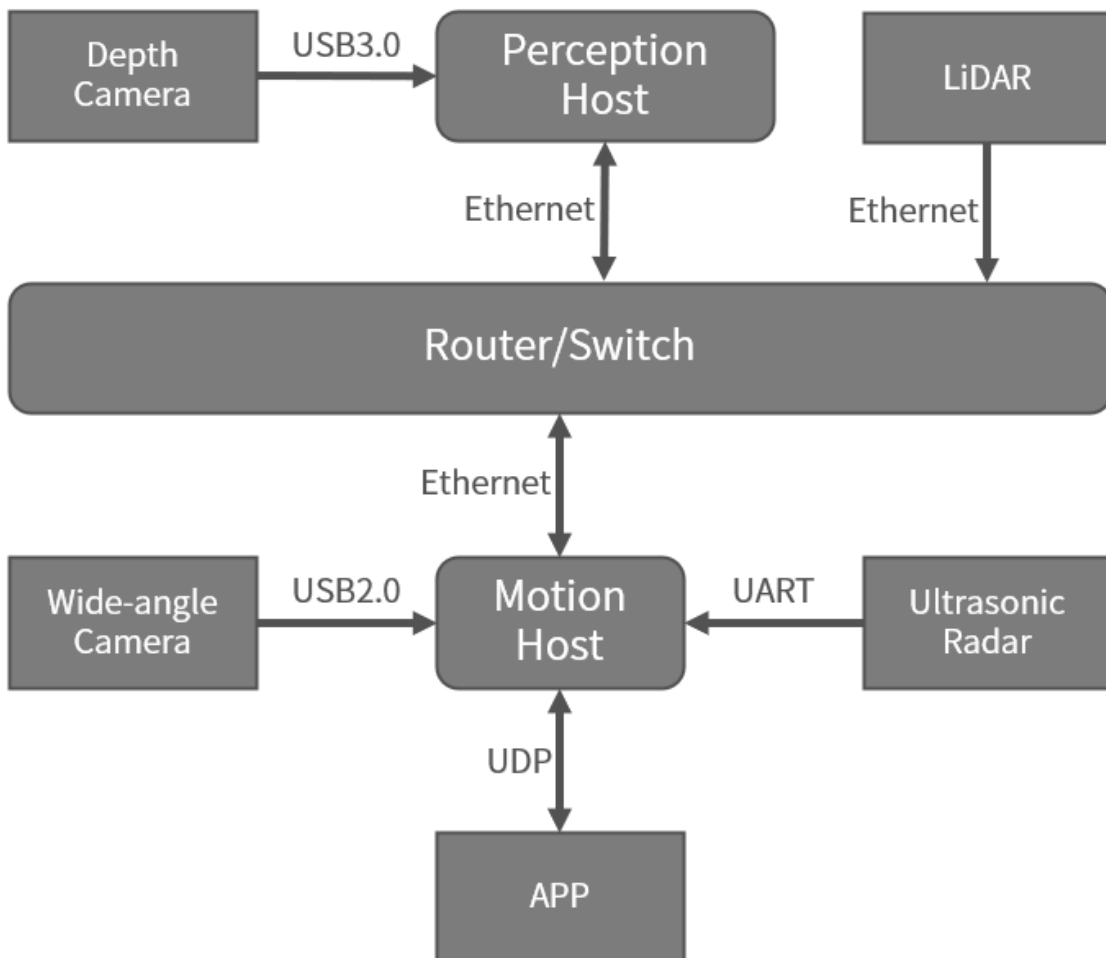
Document Description

This manual is for users who have some expertise and need to explore, develop and validate perception algorithms with Jueying Lite3. This manual version applies for Ubuntu20.

Manual Version	Update Description	Release Date
V1.0.2-0	First release	2023/6/16
V2.0.1-0	Ubuntu 20	2024/5/15
V2.1.1-0	Mapping&HDML	2024/7/26

1 Perception System

Jueying Lite3 Pro/LiDAR uses **NVIDIA Jetson Xavier NX** as its perception host for perception algorithms calculation. And the robot also provides some perception development examples to facilitate user's development.



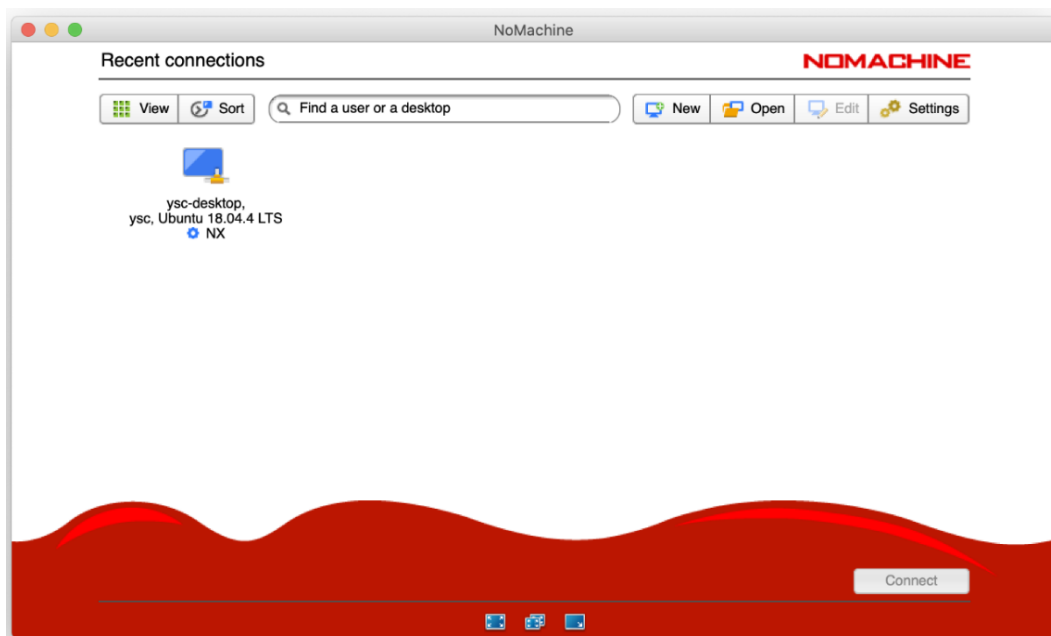
2 Preparatory Work

2.1 Remote Desktop

2.1.1 Connect

Users can remotely log in to the perception host through NoMachine.

1. Connect user's development host to the robot's WiFi.
2. Open NoMachine on development host and click "New" or "Add" to create a New connection.
 - a) Select "NX" in "Protocol" option
 - b) Enter "192.168.1.103" in the "Host" field
 - c) Select "Password" in "Authentication" option
 - d) Select "Don't use a proxy" in "Proxy" option
 - e) Leave "Save As" option default
3. Then a new remote icon will appear, as shown in the following screenshot.



- Click the icon, and enter the user name `ysc` and password `'` (a single quote), to make a remote connection.

[Caution] After logging in to the perception host, if desktop is locked or a terminal command requires you to enter a password, the password is `'` (a single quote).

2.1.2 Troubleshooting

- If the password is incorrect when you enter the password `'`, try to switch the IME to English and enter `'` again.
- If you experience a white screen after connecting to the remote desktop, please click the "Settings" button of NoMachine. When the settings page pops out, click "Server Preferences", then "Updates". Click "Check now" to update the software. After the update is complete, try to connect again. If NoMachine shows "session negotiation failed" message after entering the password, you will need to connect to the perception host via SSH from your computer and repair it.

```

1 | ssh ysc@192.168.1.103          # password is ' (a single quote)
2 | sudo su
3 | cd /usr/NX/var/db/limits/
4 | ls                          # list files in /limits
5 | rm xxxxx xxxxxx xxxxxx      # delete xxxxx xxxxxx xxxxxx

```

If still shows "session negotiation failed", repeat the operation and restart the perception host using `sudo reboot`.

2.2 Connect Perception Host Via HDMI

Jueying Lite3 Pro/LiDAR supports connection to the perception host desktop via the HDMI port on the back. The perception host automatically boots into tty3 terminal by

default. If you want to enter the GUI(Graphic user interface) automatically after the host is started, you need to make related settings.

2.2.1 Start GUI Automatically on Boot

1. First, use an HDMI cable to connect the perception host to the monitor and boot the robot. The monitor will show the boot process. If the screen shows "[OK] Started Session 1 of user ysc. ", the system starts successfully.

```
[ 25.916205] Bridge firewalling registered
[ OK ] Started User Manager for UID 1000.
[ OK ] Started Session 1 of user ysc.
```

[Caution] If the NVIDIA logo or "[OK] Started Session 1 of user ysc. " doesn't show on screen after the system starts, but "[Failed] " is displayed, the perception host hardware may be faulty. Please contact after-sales personnel for help.

2. After successfully booting the robot, use USB interface to connect keyboard, and press "Ctrl+Alt+F3" to enter the tty3 terminal.

```
Ubuntu 20.04.6 LTS lite tty3
lite login:
```

3. Then enter the user name `ysc` and password ' (a single quote) to log in.

```
Ubuntu 20.04.6 LTS lite tty3
lite login: ysc
Password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.10.120-tegra aarch64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Expanded Security Maintenance for Applications is not enabled.

222 updates can be applied immediately.
107 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

68 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
```


- After successfully logging in (pictured above), navigate to `"/usr/share/X11/xorg.conf.d"` directory and move the `"xorg.conf"` file to other directory. Specific commands are as follows:

```
1 | cd /usr/share/X11/xorg.conf.d/
2 | sudo mv xorg.conf ..           #move the xorg.conf file up to /usr/share/X11
```

And enter the password ' (single quotes) after "[sudo] password for ysc:".

```
ysc@lite:~$ cd /usr/share/X11/xorg.conf.d/
ysc@lite:/usr/share/X11/xorg.conf.d$ sudo mv xorg.conf ..
[sudo] password for ysc:
ysc@lite:/usr/share/X11/xorg.conf.d$ _
```

- After rebooting the robot, the perception host will automatically enter the GUI. If the GUI does not appear, it may be because the system has not cleared the previously configured cache. Please reboot the robot again.

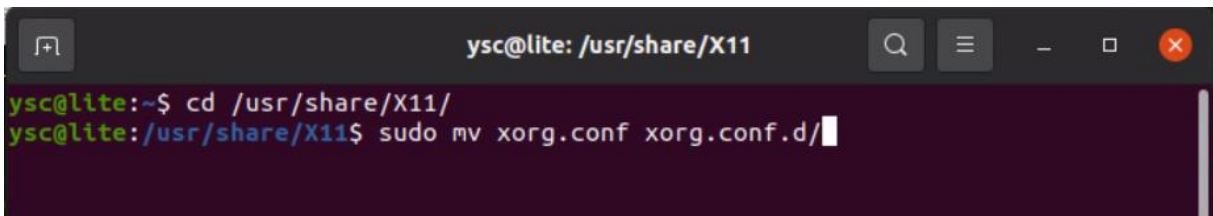
2.2.2 Start tty3 Automatically On Boot

Move `"xorg.conf"` back to the original path and reboot the robot to restore the tty3 to boot.

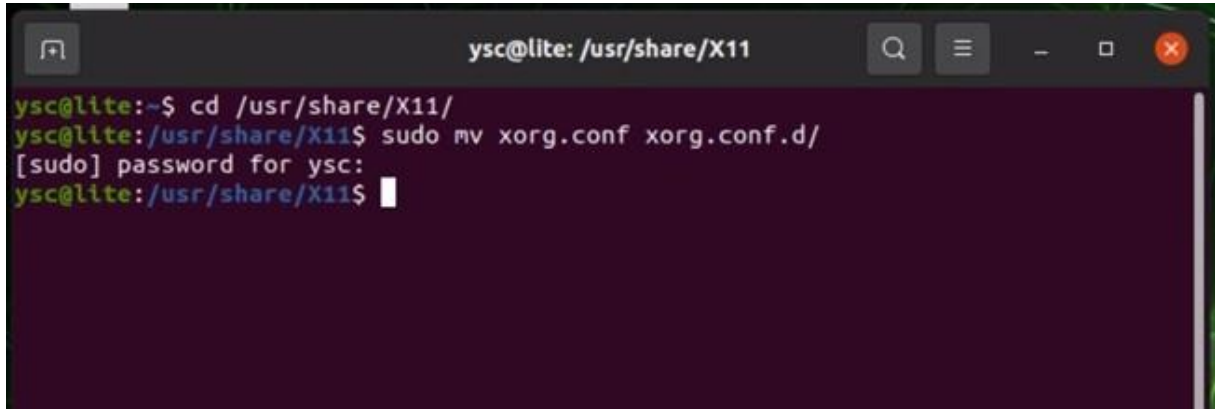
Method is as follows:

- Boot the robot, after entering the GUI, press `"Alt+Ctrl+T"` to open the terminal window and enter the following:

```
1 | cd /usr/share/X11/
2 | sudo mv xorg.conf xorg.conf.d/ # Move the xorg.conf file to xorg.conf.d/
```



2. Enter the password ' (single quotes) to move the file. If the move fails, it may be that the "xorg.conf" file is not under "/usr/share/X11/". Find the actual location of the "xorg.conf" file and move it to the "/usr/share/X11/xorg.conf.d/" directory.

A terminal window titled 'ysc@lite: /usr/share/X11' with search, menu, and window control icons. The terminal shows the following commands and output:

```
ysc@lite:~$ cd /usr/share/X11/  
ysc@lite:/usr/share/X11$ sudo mv xorg.conf xorg.conf.d/  
[sudo] password for ysc:  
ysc@lite:/usr/share/X11$
```

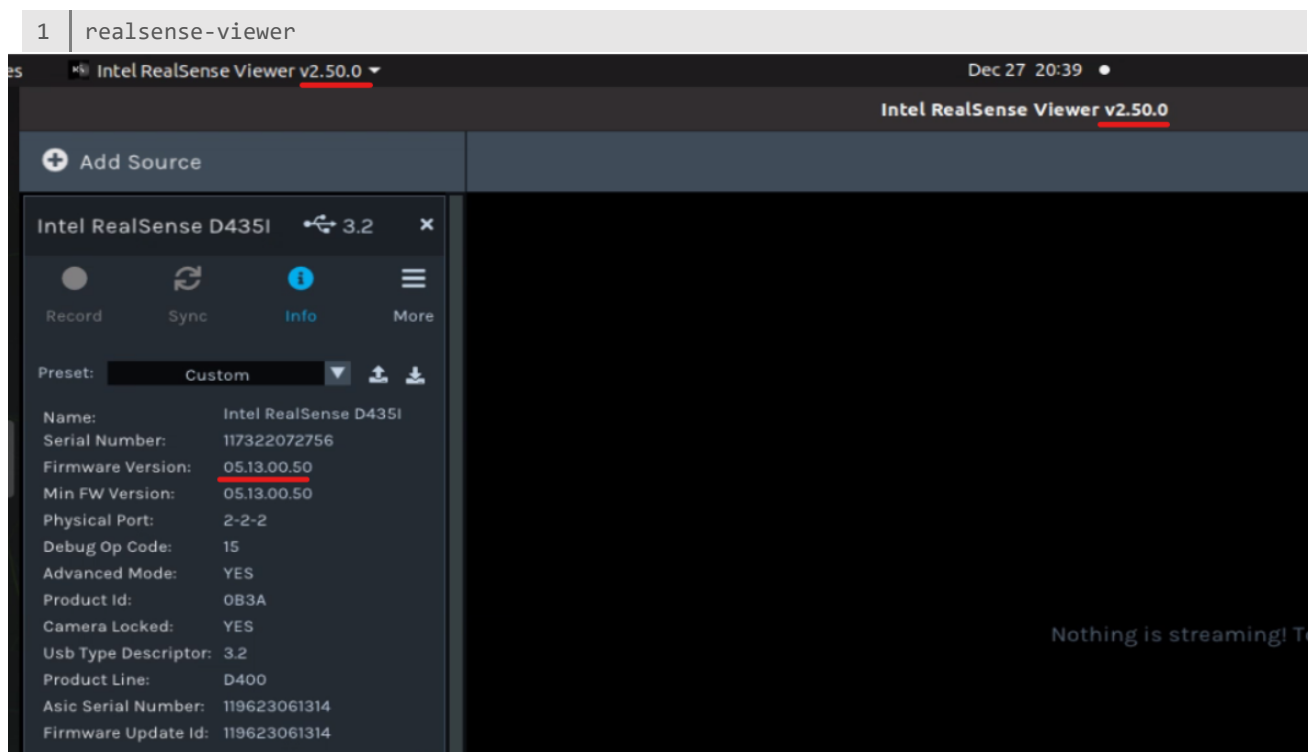
3. Reboot the robot. The perception host will enter the tty3 terminal by default while booting.

3 Depth Camera

Jueying Lite3 Pro/LiDAR is equipped with Intel RealSense D435i.

3.1 Camera Driver

The depth camera driver Intel RealSense SDK has been installed on the perception host of Jueying Lite3. Users can open the visualization tool provided by Intel by entering the following command line in the Terminal:



Click on the "Info" button for more detailed parameter information, such as the serial number, firmware version and so on.

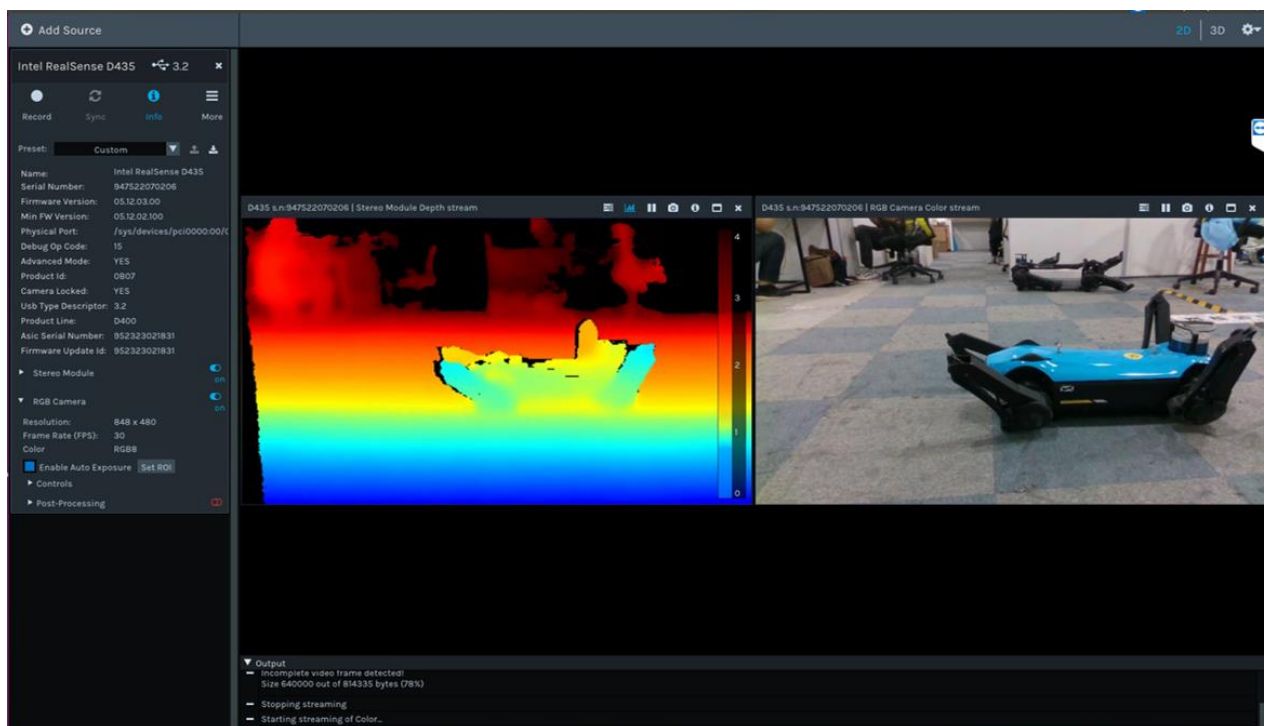
[Caution] The `realsense-ros` package depends on `librealsense` v2.50.0, which corresponds to the Depth Camera firmware version 05.13.00.50. The Version of `librealsense` is displayed in the window title, and the Firmware Version is displayed by clicking the "Info" button.

Click the triangle to Expand "Stereo Module" or "RGB Camera" and you can configure camera parameters such as resolution and frame rate.

3.2 Camera Test

Before using the camera driver for development, first check whether the depth camera is connected normally:

1. Make sure that an Intel RealSense D435i is added;
2. Click the on/off switch of the stereo module and the RGB camera. If the depth map and color map are successfully displayed, it indicates that the depth camera is properly connected.



3.3 Realsense Camera Development

The Library *librealsense* and related libraries are compiled based on CUDA and have been installed in `/usr/local/lib` and `/usr/local/include`. When development, you can include corresponding header files and link corresponding libraries for compilation.

```
ysc@lite:/usr/local/include$ ls
GLFW                librealsense2-gl    livox_lidar_cfg.h
librealsense2       livox_lidar_api.h   livox_lidar_def.h
```

```
ysc@lite:/usr/local/lib$ ls
cmake                librealsense2-gl.so.2.50    ocaml
libglfw.a            librealsense2-gl.so.2.50.0  pkgconfig
libglfw3.a           librealsense2.so            python2.7
liblivox_lidar_sdk_shared.so librealsense2.so.2.50      python3.8
liblivox_lidar_sdk_static.a librealsense2.so.2.50.0    python3.9
librealsense2-gl.so  librealsense-file.a
```

The `realsense-ros` package is located in the `/home/ysc/lite_cog/drivers/realsense_ws` directory. Related functions can be enabled through the system service by running the command: `sudo systemctl start realsense`. It will use the file `dr_camer.launch` in the `/home/ysc/lite_cog/drivers/realsense_ws/src/realsense2_camera/launch` folder to start the realsense camera. If you need to modify the startup parameters of the camera, modify the launch file.

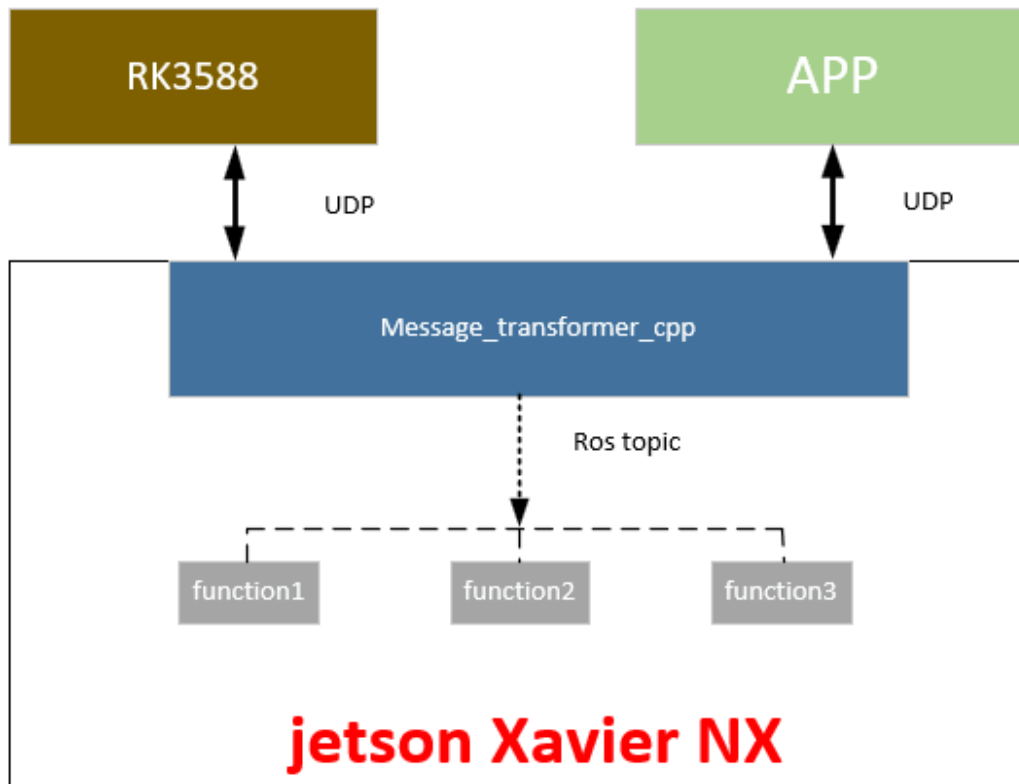
4 Message Transformer

4.1 Introduction

This package enables the conversion between ROS and UDP messages.

The data transmission between the perception host and the motion host or app is based on the UDP protocol. **Message_transformer** can be used as the following:

1. transform UDP messages sent by motion host into ROS topic messages and publish, and send motion control commands issued by perception host to motion host using UDP;
2. receive control commands from the app to turn on and off some AI functions on perception host.



ROS topics:


```

1 rostopic info xxxxxx
2 rostopic echo xxxxxx
3 # xxxxxx refers to the topic name, and users can subscribe to the topic for development

```

3. Use the topic `/cmd_vel` to send velocity commands to motion host, in the format of

`geometry_msgs/Twist`:

```

1 geometry_msgs/Vector3 linear # Linear velocity (m/s)
2 float64 x # Longitudinal velocity: positive value when going
3 forward
4 float64 y # Lateral velocity: positive value when going left
5 float64 z # Invalid parameter
6 geometry_msgs/Vector3 angular # Angular velocity (rad/s)
7 float64 x # Invalid parameter
8 float64 y # Invalid parameter
9 float64 z # Angular velocity: positive value when turning left

```

- a) Users can publish to this topic in C++ or Python programs compiled based on ROS (refer to <http://wiki.ros.org/ROS/Tutorials> for learning about ROS). Users can also publish messages to the topic for debugging in terminal. Please first type the following codes in terminal:

```
1 rostopic pub /cmd_vel geometry_msgs/Twist
```

- b) Before pressing Enter, add a space after the codes and press Tab key to automatically complement the message type as follows:

```

1 rostopic pub /cmd_vel geometry_msgs/Twist "linear:
2 x: 0.0
3 y: 0.0
4 z: 0.0
5 angular:
6 x: 0.0
7 y: 0.0
8 z: 0.0
9 "

```

- c) Use the left/right arrow keys on the keyboard to move the cursor, modify the velocity values, and then add `-r 10` after `geometry_msgs/Twist` to specify the posting frequency (10Hz) as follows:


```

1  rostopic pub /cmd_vel geometry_msgs/Twist -r 10 "linear:
2  x: 0.2
3  y: 0.1
4  z: 0.0
5  angular:
6  x: 0.0
7  y: 0.0
8  z: 0.3
9  "
```

- d) Press Enter key to run and publish the topic messages.
- e) Message_transformer can subscribe to this topic, transform the topic messages into UDP messages and send them to motion host.
- f) After the transmission process is normally opened, make the robot stand up and start the auto mode in the APP Settings page, and the robot can act at the above speed.

[Caution] Please debug in an open area to prevent damage to people or objects. In case of an emergency, press the STOP button in time, or turn off the auto mode.

4.3 Development

4.3.1 Package Structure

```

/home/ysc/lite_cog/transfer/src
├─ CMakeLists.txt
├─ message_transformer
│   └─ CMakeLists.txt
│       └─ include
│           ├── protocol.h
│           └─ sensor_logger.h
│               └─ launch
│                   └─ message_transformer.launch
├─ msg
│   ├── SimpleCMD.msg
│   └─ ComplexCMD.msg
├─ package.xml
└─ src
```

```

├─ nx2app.cpp
├─ qnx2ros.cpp
├─ ros2qnx.cpp
└─ sensor_checker.cpp

```

1. *nx2app.cpp* is mainly used for UDP communication between perception host and app. The app sends command code to perception host and *nx2app.cpp* will execute tasks according to the received command. The commands sent by app are structured as follows:

```

1  class SimpleCMD{
2  public:
3      int32_t cmd_code;
4      int32_t cmd_value;
5      int32_t type;
6  };

```

2. *qnx2ros.cpp* is used to receive the data sent by motion host and transform it into ROS topic messages.

```

1  leg_odom_pub_ = nh.advertise<geometry_msgs::PoseWithCovarianceStamped>("leg_odom", 1);
2  leg_odom_pub2_ = nh.advertise<nav_msgs::Odometry>("leg_odom2", 1);
3  joint_state_pub_ = nh.advertise<sensor_msgs::JointState>("joint_states", 1);
4  imu_pub_ = nh.advertise<sensor_msgs::Imu>("/imu/data", 1);
5  handle_pub_ = nh.advertise<geometry_msgs::Twist>("/handle_state", 1);
6  ultrasound_pub_ = nh.advertise<std_msgs::Float64>("/us_publisher/ultrasound_distance",
1);

```

3. *ros2qnx.cpp* can subscribe to the topic published by other nodes, transform the messages into UDP data and send them to motion host.

```

1  ros::Subscriber vel_sub = nh.subscribe("cmd_vel", 1, &ROS2QNX::CmdVelCallback,
2  &ros2qnx);
3  ros::Subscriber vel_sub2 = nh.subscribe("cmd_vel_corrected", 1,
4  &ROS2QNX::CmdVelCallback, &ros2qnx);
5  ros::Subscriber simplecmd_sub = nh.subscribe("simple_cmd", 1,
6  &ROS2QNX::SimpleCMDCallback, &ros2qnx);
7  ros::Subscriber complexcmd_sub = nh.subscribe("complex_cmd", 1,
&ROS2QNX::ComplexCMDCallback, &ros2qnx);

```

5 People Tracking

5.1 Introduction

This case first utilizes DeepStream, YOLOv8 and TensorRT to recognize and track the target individuals in the scene and then calculates the target position and transmits it to the motion host to enable the robot to follow the target people. Hardware decoding based on DeepStream is used to obtain an h264-encoded 720p resolution rtsp video stream, and TensorRT is used to accelerate the YOLOv8 human detection model to recognize people in open scenes, enabling it to recognize people at approximately 20 fps and track them at around 10 fps.

This case is divided into two parts: recognition and tracking.

1. Recognition algorithm performs deep learning neural network for visual recognition to find the position of human body in the picture. When multiple bodies appear in the picture, all the human bodies in the frame are first recognized. Then, the features of the human body identified in each frame of video are extracted based on deep learning and compared one by one to determine the trajectory of the same person in the previous and subsequent frames.
2. Tracking algorithm allows users to choose the target human they want to follow in the screen. The robot can achieve real-time targeting and continuous tracking. The recognition algorithm can determine the direction and distance of people from the robot so that the robot can respond accordingly (translate or rotate). Its velocity can

also be adjusted in real-time depending on the distance between the robot and the person being tracked.

- a) Too close: When the target is too close, the robot will stop to prevent a collision.
- b) Close: When the target is close, the robot will dynamically slow down in real-time to get close to the target.
- c) Far: The robot will move at maximum speed when the target is far away.

The source codes of yolov8 and sdk_hub used in this case are from [ultralytics](#) and [hub-sdk](#). Also you can search materials about yolov8 on the Internet.

5.2 Usage

[Caution] When the program is started, the initialization of video decoding and deep learning inference environment are required, which takes about 40s. If the function cannot be started for a long time, connect the controller to the robot to check whether the video stream works properly.

1. Open a Terminal and enter the following command to start the program:

```
1 | cd /home/ysc/lite_cog/track/src
2 | python3 run_tracker.py
```

2. Use the app to make the robot stand up and start the auto mode.
3. When people appear, the system will assign numbers to all the people who has been identified and displayed the numbers on the screen. Use the keyboard to enter the assigned number of the person you want to follow and press enter to confirm.

[Caution] When entering a target number, kindly ensure that the video window is on top.

4. The robot will then track and identify the target.

5. You can press Enter key to reset the target when tracking, or when the target is lost and "Miss Object" is displayed.
6. Press Esc to end the program.

5.3 Development

The package , *people_tracking*, provided in this case is in /home/ysc/lite_cog/track.

5.3.1 Package Structure

```
/home/ysc/lite_cog/track
├─ model
│   ├── export_engine.sh
│   ├── yolov8n_amd.engine
│   ├── yolov8n_arm.engine
│   ├── yolov8n.onnx
│   └─ yolov8n.pt
└─ src
    ├── GStreamerWrapper
    │   └─ GStreamerWrapper.py
    ├── hub_sdk
    ├── RobotController
    │   ├── FpsCounter
    │   │   └─ FpsCounter.py
    │   ├── RobotController.py
    │   ├── ROSTransfer
    │   │   ├── ROS1Transfer.py
    │   │   ├── ROS2Transfer.py
    │   │   └─ TransferConstants.py
    │   └─ YoloWrapper
    │       ├── CocoTypeId.py
    │       └─ YoloWrapper.py
    ├── run_tracker.py
    ├── test
    │   ├── pull.py
    │   ├── pull.sh
    │   └─ yolov8.py
    └─ ultralytics
```

1. run_tracker.py is the main program.

2. GStreamerWrapper is a DeepStream-based GStreamer hardware decoder used to obtain RTSP video streams.
3. The main operation logic of RobotController.py is reflected in its Run() function, which is used to identify the human body in the image obtained from the video stream, and then send motion instructions.
4. ultralytics is an open source YOLOv8 program package, which is used to reason and track image frames obtained from video streams, and is the operation dependency of YOLOWrapper in RobotController. The ultralytics/cfg folder is the storage address of various YOLOv8 configuration files. Each configuration file has been fully commented.
5. sdk_hub is the open source sdk_hub program package, which is the operation dependency of the YOLOv8 package.

6 LiDAR-based SLAM and Navigation

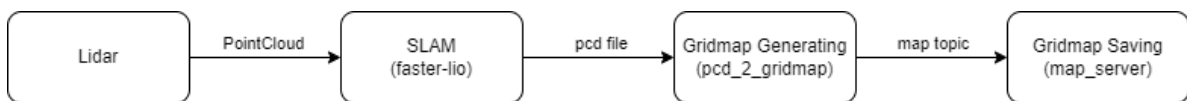
This case uses LiDAR and imu to achieve mapping (indoor and outdoor scenes), localization, navigation, and obstacle avoidance on perception host. The robot can achieve real-time localization and online 3D mapping. When localizing in a map, by fusing IMU, it will not lose its location due to falling or high-speed rotation. Map-based navigation is achieved using the *move_base* package.

[Caution] Before mapping, please check "`~/Desktop/version_log.txt`" document. If the version is v3.1.04 or later, please refer to 6.1 for mapping. If the version is earlier than v3.1.04, please refer to 6.2 for mapping.

6.1 Mapping(for v3.1.04 and later)

6.1.1 Introduction

This case uses [SLAM Mapping](#) released on Github by Dr. Gao Xiang's team. The main operation process and data flow diagram are shown as below:



The mapping package is located in the "`~/lite_cog/slam/src`" path and contains three packages: *faster-lio*, *pcd_2_gridmap* and *map_server*. The *faster-lio* package is responsible for building 3D point cloud maps (.pcd), The *pcd_2_gridmap* package is responsible for converting 3D point cloud maps (.pcd) to grid maps (.pgm) and publishing them. The *map_server* package is responsible for saving grid maps (.pgm).

6.1.2 Usage

[Caution] Before mapping, please check whether there is a previously created map in the /home/ysc/lite_cog/system/map folder. If so, you can move it to another folder to avoid overwriting.

[Caution] Mapping requires more computing resources, so please turn off all the AI options on the app first.

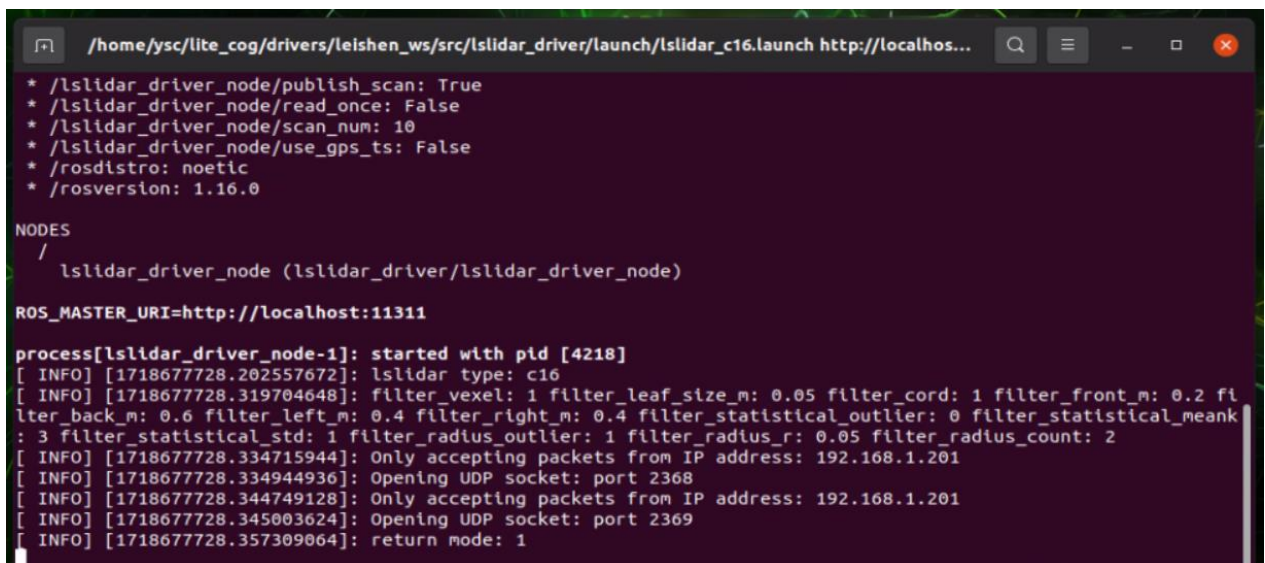
1. Open the Terminal and enter the following to start the LiDAR driver:

```
1 | cd /home/ysc/lite_cog/system/scripts/lidar
2 | ./start_lsldar.sh
```

If the LiDAR driver node fails to start, check whether the LiDAR has connected to the perception host using the following command:

```
1 | ping 192.168.1.201
```

[Caution] This terminal should be kept running during mapping.



```
/home/ysc/lite_cog/drivers/leishen_ws/src/lsldar_driver/launch/lsldar_c16.launch http://localhost:11311
* /lsldar_driver_node/publish_scan: True
* /lsldar_driver_node/read_once: False
* /lsldar_driver_node/scan_num: 10
* /lsldar_driver_node/use_gps_ts: False
* /roscdistro: noetic
* /rosversion: 1.16.0

NODES
/
  lsldar_driver_node (/lsldar_driver/lsldar_driver_node)

ROS_MASTER_URI=http://localhost:11311

process[lsldar_driver_node-1]: started with pid [4218]
[ INFO] [1718677728.202557672]: lsldar type: c16
[ INFO] [1718677728.319704648]: filter_voxel: 1 filter_leaf_size_m: 0.05 filter_cord: 1 filter_front_m: 0.2 fl
lter_back_m: 0.6 filter_left_m: 0.4 filter_right_m: 0.4 filter_statistical_outlier: 0 filter_statistical_meank
: 3 filter_statistical_std: 1 filter_radius_outlier: 1 filter_radius_r: 0.05 filter_radius_count: 2
[ INFO] [1718677728.334715944]: Only accepting packets from IP address: 192.168.1.201
[ INFO] [1718677728.334944936]: Opening UDP socket: port 2368
[ INFO] [1718677728.344749128]: Only accepting packets from IP address: 192.168.1.201
[ INFO] [1718677728.345003624]: Opening UDP socket: port 2369
[ INFO] [1718677728.357309064]: return mode: 1
```

2. Start the mapping program:
 - a) The script *start_slam.sh*, which starts the mapping program, is in the path /home/ysc/lite_cog/system/scripts/slam and reads as follows:

```
1 | #!/bin/sh
2 |
3 | # Open the mapping program
```



```

4  gnome-terminal -x bash -c "source /home/ysc/lite_cog/slam/devel/setup.bash;
5  roslaunch faster_lio mapping_c16.launch; read -p 'Press any key to exit...'"
6
7  # open a terminal used for creating grid_map
8  gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/gridmap.sh;
9  read -p 'Press any key to exit...'"
10
11 # open a terminal used for saving grid map
12 gnome-terminal -x bash -c "bash
13 /home/ysc/lite_cog/system/scripts/slam/save_map.sh; read -p 'Press any key to
   exit...'"

```

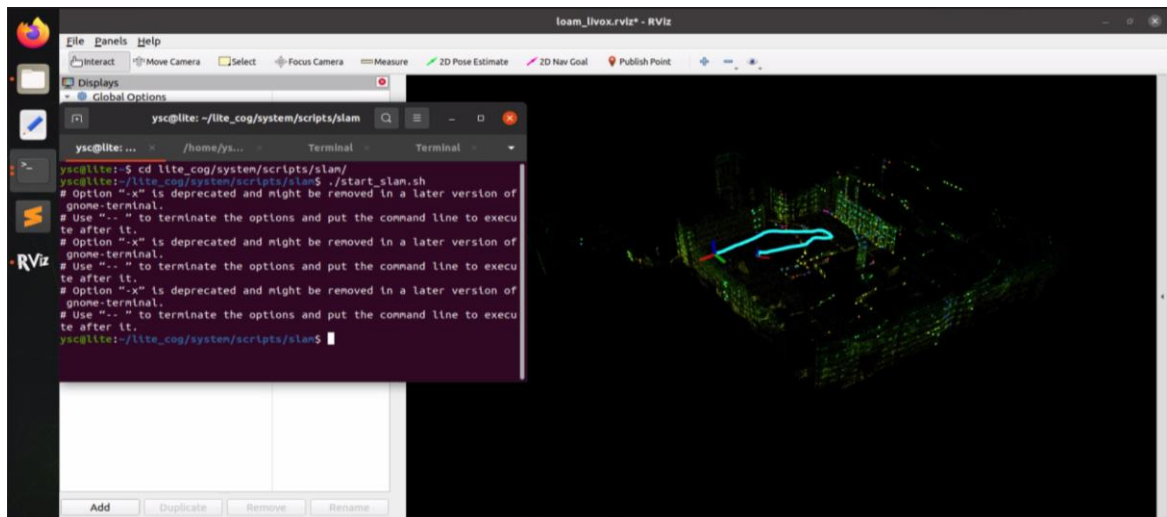
- b) After logging into the perception host desktop using NoMachine and remotely controlling the robot to stand up, open a Terminal and enter the following command to start the mapping program using the script:

```

1  cd /home/ysc/lite_cog/system/scripts/slam
2  ./start_slam.sh

```

- c) After executing the above command, the visualization tool RViz will be launched, and three terminal tabs will be generated in the terminal running the script `start_slam.sh`, respectively, to run *fast-lio*, generate grid map and save grid map.



3. Operate the robot and guide it around the designated area to construct the map. When taking turns, please slow down. Also, be mindful of LiDAR's blind spots and keep the robot at a minimum distance of 0.5 meters from any walls.

4. After finishing scanning the designated area, check whether the point cloud map matches the real environment in RViz.
5. Find the corresponding tab page of faster-lio program after completing the map scanning, press "Ctrl+C" to stop mapping, the program will automatically save the 3D point cloud file (.pcd) to the ~/lite_cog/system/map directory, and display the average processing time (time is for reference only). Press Enter to close this tab page.

```

/home/ysc/lite_cog/slam/src/faster-lio/launch/mapping_c16.launch http://localhost:11311
ysc@lite: ~/lite_cog/system/sc...  /home/ysc/lite_cog/slam/src/f...  Terminal  Terminal
I0618 10:42:59.238492 13321 laser_mapping.cc:331] [ mapping ]: In num: 727 downsamp 331 Map grid num: 3240 effect num : 321
I0618 10:42:59.309013 13321 laser_mapping.cc:331] [ mapping ]: In num: 758 downsamp 334 Map grid num: 3240 effect num : 328
^C[rviz-2] killing on exit
[laserMapping-1] killing on exit
[ WARN ] [1718678579.411170120]: catch sig 2
I0618 10:42:59.437414 13321 laser_mapping.cc:331] [ mapping ]: In num: 702 downsamp 318 Map grid num: 3240 effect num : 308
I0618 10:42:59.437973 13321 run_mapping_online.cc:43] finishing mapping
I0618 10:42:59.438230 13321 laser_mapping.cc:865] current scan saved to /home/ysc/lite_cog/system/map/lite3.pcd
I0618 10:42:59.547519 13321 laser_mapping.cc:869] finish done
I0618 10:42:59.547705 13321 utils.h:52] >>> ===== Printing run time =====
I0618 10:42:59.547789 13321 utils.h:54] > [ Ivox Add Points ] average time usage: 0.0595673 ms , called times: 556
I0618 10:42:59.547889 13321 utils.h:54] > [ Incremental Mapping ] average time usage: 0.260003 ms , called times: 556
I0618 10:42:59.547972 13321 utils.h:54] > [ ObsModel (IEKF Build Jacobian) ] average time usage: 0.389098 ms , called times: 2194
I0618 10:42:59.548059 13321 utils.h:54] > [ ObsModel (Lidar Match) ] average time usage: 2.21896 ms , called times: 2194
I0618 10:42:59.548146 13321 utils.h:54] > [ Downsample PointCloud ] average time usage: 0.228894 ms , called times: 556
I0618 10:42:59.548224 13321 utils.h:54] > [ IEKF Solve and Update ] average time usage: 12.7271 ms , called times: 556
I0618 10:42:59.549230 13321 utils.h:54] > [ Preprocess (Standard) ] average time usage: 0.391041 ms , called times: 560
I0618 10:42:59.551244 13321 utils.h:54] > [ Undistort Pcl ] average time usage: 1.97235 ms , called times: 553
I0618 10:42:59.551353 13321 utils.h:59] >>> ===== Printing run time end =====
I0618 10:42:59.551426 13321 run_mapping_online.cc:47] save trajectory to: ./Log/traj.txt
I0618 10:42:59.586180 13321 laser_mapping.h:34] laser mapping deconstruct
shutting down processing monitor...
... shutting down processing monitor complete
done
Press enter to exit...

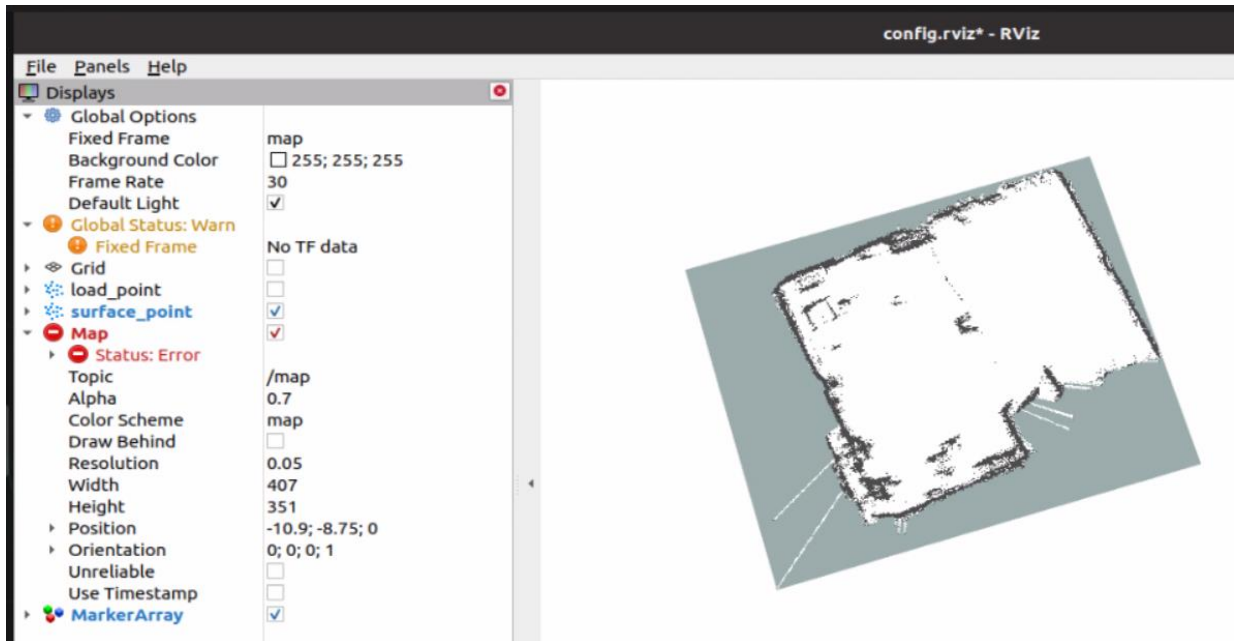
```

6. After saving the 3D point cloud file (.pcd) successfully, find the tab page as shown in the following figure, enter 1 and press Enter key. After a while, *pcd_2_gridmap* package will be called to convert the point cloud map into a grid map, and the next step can be carried out when the RViz window pops up and displays the grid map.

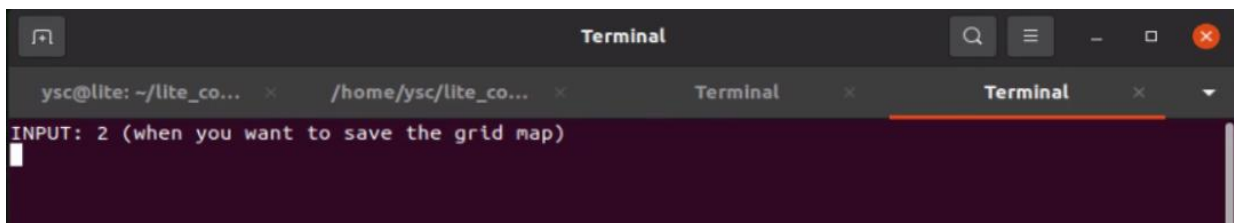
```

Terminal
ysc@lite: ~/...  /home/ysc/...  Terminal  Terminal
INPUT: 1 (when you want to creat the grid map)

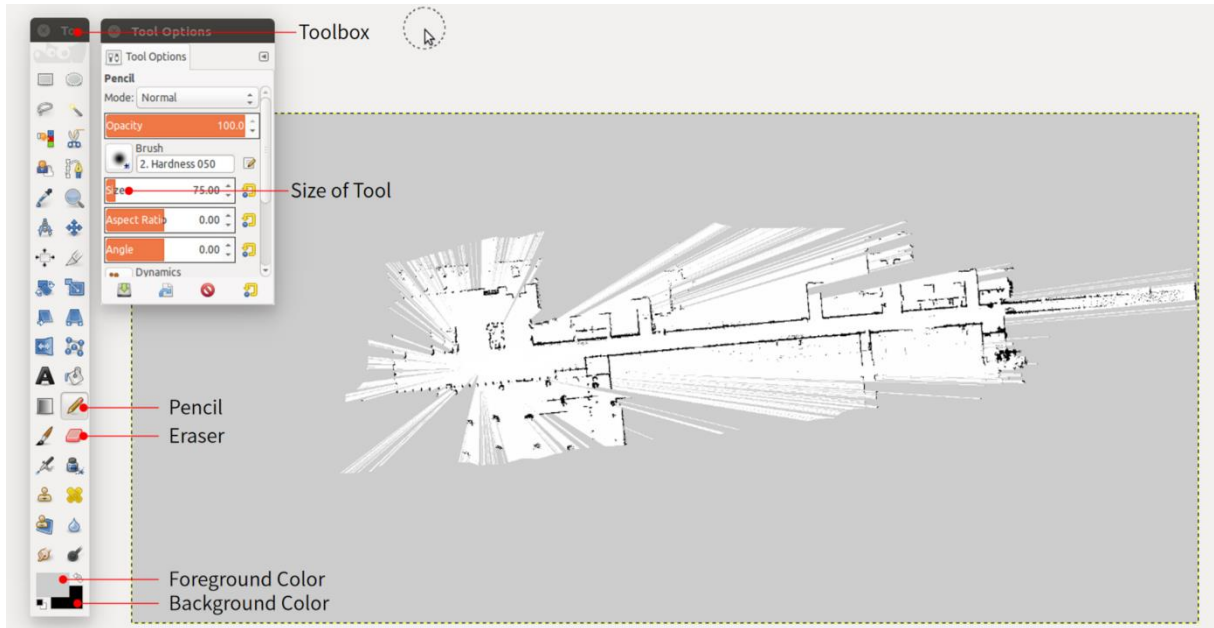
```



7. To save the grid map, first, select the Terminal that displays "when you want to save the grid map". Next, enter the number 2 and press Enter to call [map_server](#). After that, map files will be saved to /home/ysc/lite_cog/system/map, including the .yaml file, .pgm file, and .pcd file.



8. If the grid map (.pgm) is not completely in line with the actual environment or users need to manually delimit passable areas, GIMP Image Editor can be used to edit it. Open a Terminal and type `gimp` to open GIMP Image Editor and drag the grid map (.pgm) into it.



- a) *Toolbox* can be opened by choosing [Windows] – [New Toolbox] in the top menu bar if it is not displayed.
 - b) *Foreground Color* specifies the color of *Pencil* and *Background Color* specifies the color of *Eraser*. In the grid map, the black area is not passable, the white area is passable and the gray area is unknown. Users can erase the noise or add a virtual wall with *Pencil* or *Eraser*.
 - c) Save the modified map by clicking [File] – [Overwrite usr_map.pgm] to cover the origin file and it is not necessary to save it again when closing the editor.
9. Please close all Terminals with ctrl+c after completing all operations to avoid affecting subsequent processes.
 10. The map files will by default be saved in /home/ysc/lite_cog/system/map directory. If the path or name of map files is changed, configure in *local_rslidar_imu.launch* file located at /home/ysc/lite_cog/nav/src/hdl_localization/launch, so that localization and navigation program can call the map correctly.

```

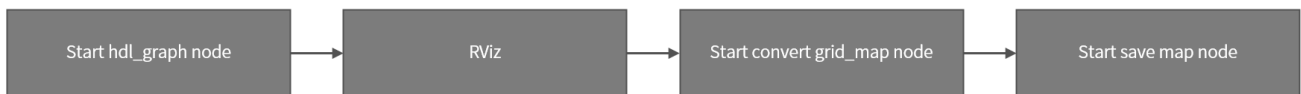
1 <arg name="map_name" default="lite3" />      //Define Map File Name
2 ...
3 <node name="MapServer" pkg="map_server" type="map_server"
  args="/home/ysc/lite_cog/system/map/$(arg map_name).yaml"/>
4
5 ...
6 ...
7 <param name="globalmap_pcd" value="/home/ysc/lite_cog/system/map/$(arg map_name).pcd"
8 />
  ...

```

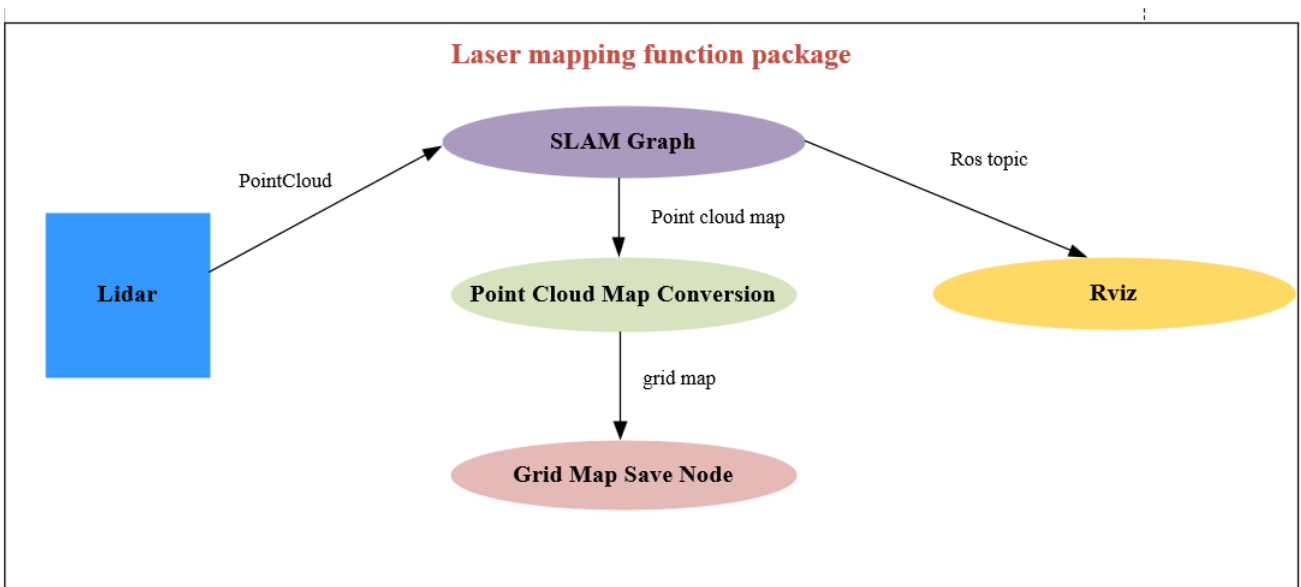
6.2 Mapping(for earlier than v3.1.04)

6.2.1 Introduction

This case uses [6DOF SLAM](#) released on Github by Kenji Koide from Toyohashi University of Technology. The main operation process is shown below:



The corresponding data flow diagram is shown below:



6.2.2 Package Structure

```
/home/ysc/lite_cog/slam
```

```
├─ build
├─ devel
├─ src
│   ├─ CMakeLists.txt
│   ├─ fast_gicp
│   ├─ hdl_graph_slam
│   ├─ map_server
│   └─ ndt_omp
└─ version
```

The *hdl_graph_slam* package builds the map.

6.2.3 Usage

[Caution] Before mapping, please check whether there is a previously created map in the `/home/ysc/lite_cog/system/map` folder. If so, you can move it to another folder to avoid overwriting.

[Caution] Mapping requires more computing resources, so please turn off all the AI options on the app first.

1. Open the Terminal and enter the following to start the LiDAR driver:

```
1 | cd /home/ysc/lite_cog/system/scripts/lidar
2 | ./start_lsliidar.sh
```

If the LiDAR driver node fails to start, check whether the LiDAR has connected to the perception host using the following command:

```
1 | ping 192.168.1.201
```

2. Start the mapping program:

- a) The script *start_slam.sh*, which starts the mapping program, is in the path `/home/ysc/lite_cog/system/scripts/slam` and reads as follows:

```
1 | #!/bin/sh
2 |
3 | # open rviz
4 | gnome-terminal -x bash -c "cd /home/ysc/lite_cog/slam; source devel/setup.bash;
5 | roslaunch hdl_graph_slam mapping_rslidar_indoor.launch;"
6 |
7 | # open rviz
```

```

8 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/rviz.sh"
9 |
10 | # open a terminal used for creating grid_map
11 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/gridmap.sh"
12 |
13 | # open a terminal used for saving map
14 | gnome-terminal -x bash -c "bash /home/ysc/lite_cog/system/scripts/slam/save_map.sh"
15 |

```

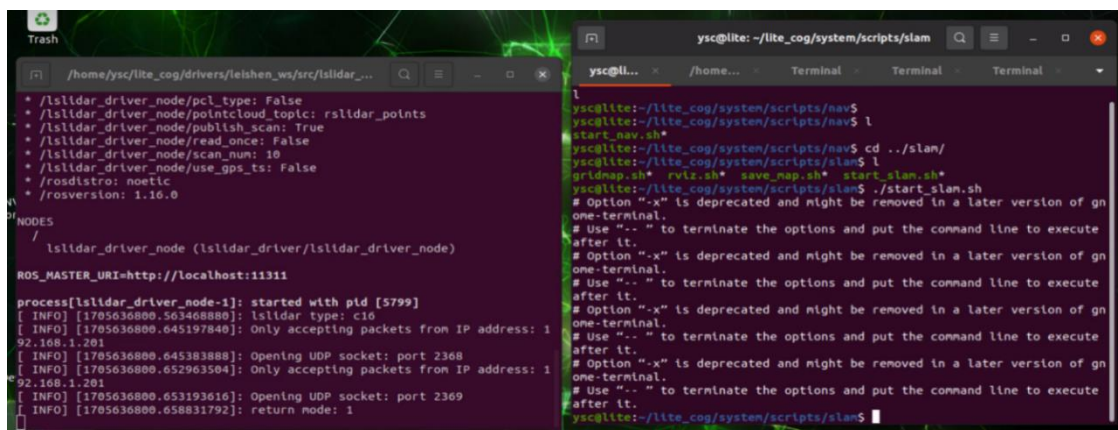
- b) After logging into the perception host desktop using NoMachine and remotely controlling the robot to stand up, open a Terminal and enter the following command to start the mapping program using the script:

```

1 | cd /home/ysc/lite_cog/system/scripts/slam
2 | ./start_slam.sh

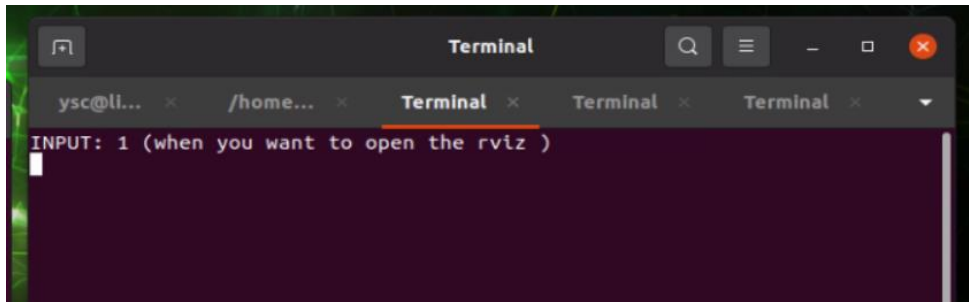
```

- c) After executing the above command, five terminal Windows will be generated, respectively used for running the scripts, running the mapping program, opening Rviz, creating grid map, and saving map. (In the picture below, the left side is the LiDAR driver window, the right side is the mapping script window) :

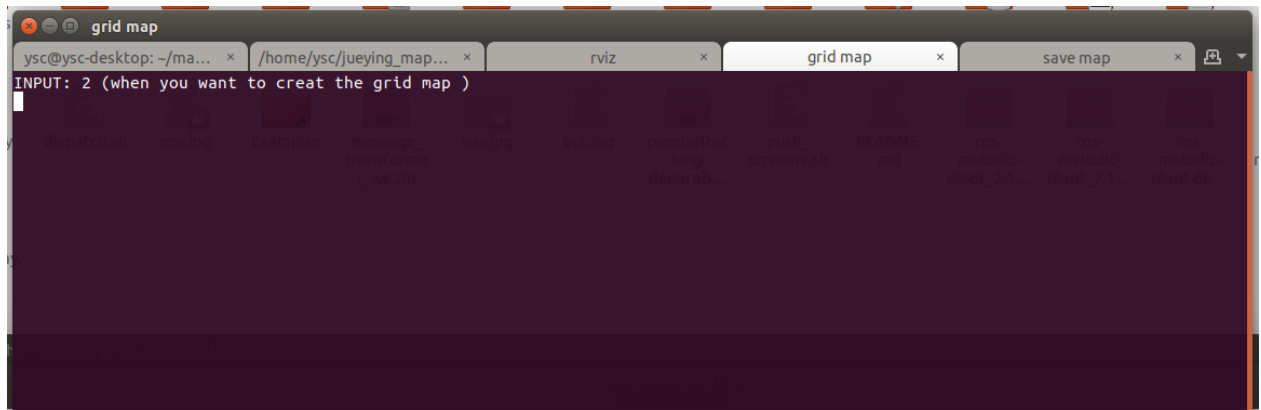


3. If you want to see the effect of mapping in real time, find the terminal used for opening RViz (as shown below), input the number 1 and press Enter, then the RViz visualization interface will be opened. Opening this interface will reduce the performance of mapping, and if you are not satisfied with the effect of mapping,

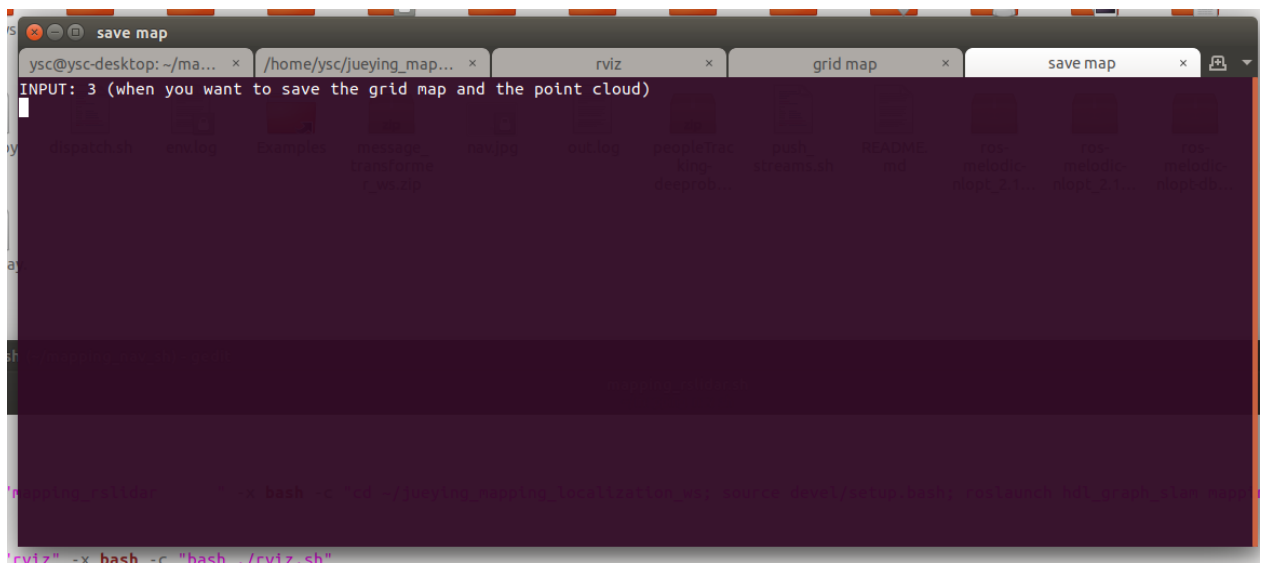
please try not to open Rviz and close the NoMachine remote interface when mapping to save computing resources).



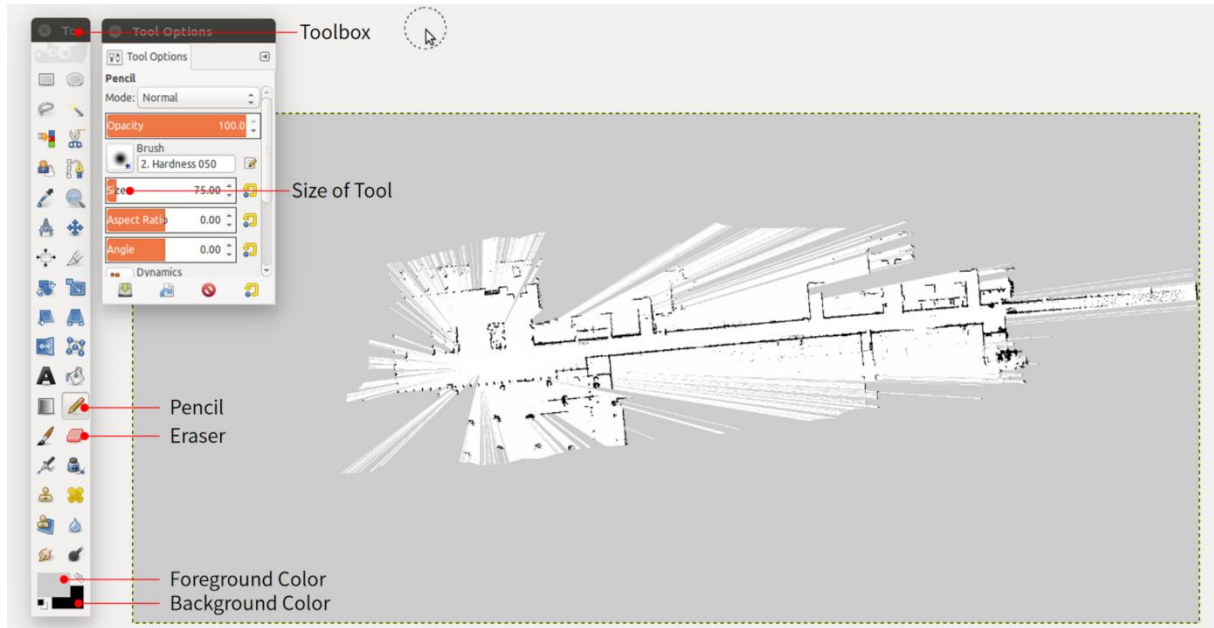
4. Operate the robot and guide it around the designated area to construct the map.
When taking turns, please slow down. Also, be mindful of LiDAR's blind spots and keep the robot at a minimum distance of 0.5 meters from any walls.
5. After finishing scanning the designated area, check whether the point cloud map matches the real environment (if the RViz has not been opened before, open it at this time). If the area is large or there is a closed loop in the real environment (such as circling around a house), please check the map whether it is a closed loop consistent with the real environment. If it is not a closed loop, you can circle again to complete the loop-closure detection.
6. To convert the point cloud map into a grid map after completing the map scanning, first, select the Terminal that displays "when you want to create the grid map" as shown in the figure below. Then, enter the number 2 and press Enter to call [octomap](#). After this, remotely control the robot to walk a short distance. Doing so will convert the point cloud map into a grid map.



7. To save the grid map, first, select the Terminal that displays "when you want to save the grid map and the point cloud". Next, enter the number 3 and press Enter to call [map_server](#). After that, remotely control the robot to walk a short distance. Map files will be saved to /home/ysc/lite_cog/system/map, including the .yaml file, .pgm file, and .pcd file.



8. If the grid map (.pgm) is not completely in line with the actual environment or users need to manually delimit passable areas, GIMP Image Editor can be used to edit it. Open a Terminal and type `gimp` to open it and drag the grid map (.pgm) into GIMP Image Editor.



- a) *Toolbox* can be opened by choosing [Windows] – [New Toolbox] in the top menu bar if it is not displayed.
 - b) *Foreground Color* specifies the color of *Pencil* and *Background Color* specifies the color of *Eraser*. In the grid map, the black area is not passable, the white area is passable and the gray area is unknown. Users can erase the noise and add a virtual wall with Pencil or Eraser.
 - c) Save the modified map by clicking [File] – [Overwrite usr_map.pgm] to cover the origin file and it is not necessary to save it again when closing the editor.
9. Please close all Terminals with ctrl+c after completing all operations to avoid affecting subsequent processes.
 10. The map files will by default be saved in /home/ysc/lite_cog/system/map directory. If the path or name of map files is changed, configure in *local_rslidar_imu.launch* file located at /home/ysc/lite_cog/nav/src/hdl_localization/launch, so that localization and navigation program can call the map correctly.

```
1 <arg name="map_name" default="lite3" />      //Define Map File Name
2 ...
3 <node name="MapServer" pkg="map_server" type="map_server"
  args="/home/ysc/lite_cog/system/map/$(arg map_name).yaml"/>
4
5 ...
6 ...
7 <param name="globalmap_pcd" value="/home/ysc/lite_cog/system/map/$(arg map_name).pcd"
8 />
9 ...
```

6.3 Localization & Navigation

This case is based on LiDAR and IMU to implement localization and navigation. The localization algorithm used in this case is [hdl_localization_algorithm](#).

[Caution] The LIDAR drive needs to be running during localization and navigation (refer to 6.1.2).

6.3.1 Usage of Point-to-point Navigation

1. Open a Terminal and enter the following codes to run the script of localization and navigation:

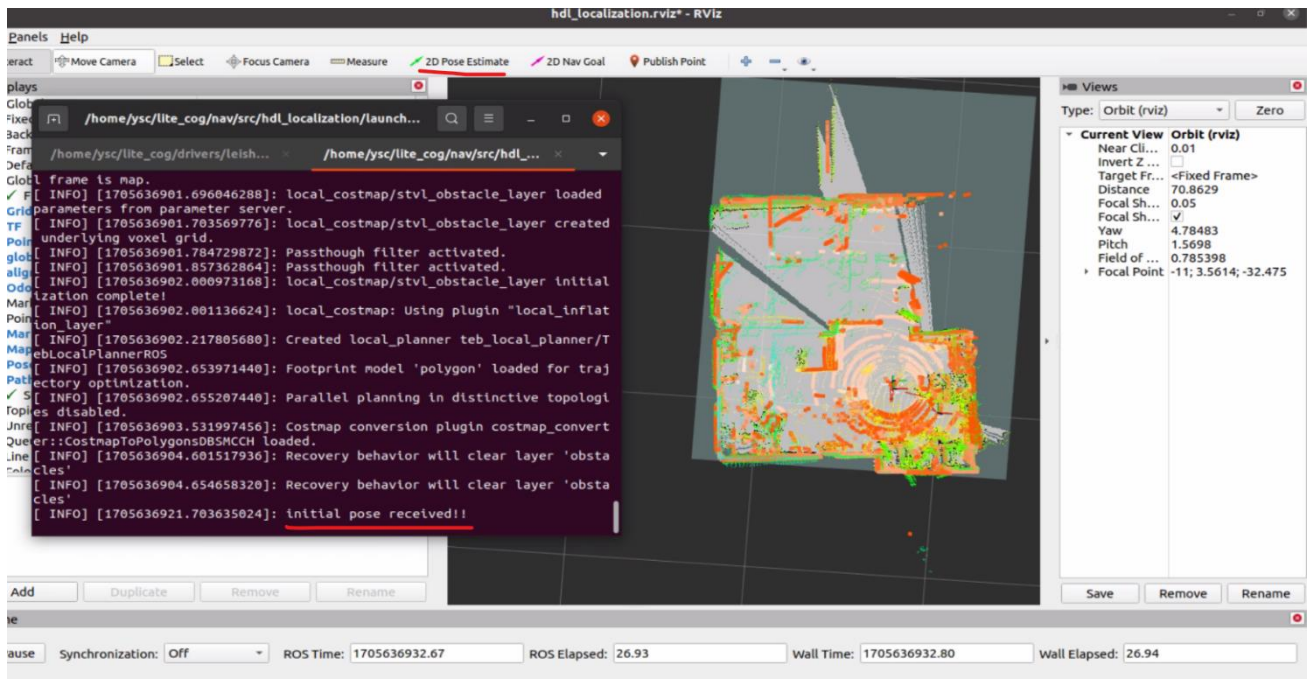
```
1 cd /home/ysc/lite_cog/system/scripts/lidar
2 ./start_lsldidar.sh
```

2. Open a terminal and enter the following command to start the node.

```
1 cd /home/ysc/lite_cog/system/scripts/nav
2 ./start_nav.sh
```

3. After RViz is opened, initialize the robot location:
 - a) Click the "2D Pose Estimate" button in the top toolbar.
 - b) According to the actual location and orientation of the robot, press the mouse left button and drag to pull out an arrow at the corresponding location on the grid map.

- c) If the positioning initialization is successful, the laser point cloud and grid map will coincide, and the terminal print "initial pose received!!" ;
- d) If the laser point cloud does not coincide with the grid map, the initial position is not correct, please re-operate;
- e) If the point cloud does not appear on the map and the terminal prints "globalmap has not been received!", please close the program with ctrl+c and try again.
- f) The base_link coordinate system is the robot coordinate system, and the x-axis (red) indicates the robot orientation.



[RViz Usage Tips] To manipulate the map on RViz, you can zoom in and out by using the mouse wheel. For rotation, you can drag the left mouse button. Meanwhile, to pan and drag the map, you need to hold Shift key and drag the left mouse button.

4. After initializing the location, a target point can be given according to a similar method:

- a) Click the "2D Nav Goal" button in the top toolbar.

- b) Press the left mouse button and drag on the grid map to specify a navigation goal and its orientation.
 - c) If successfully specifying a navigation goal, the planned path will be computed and shown. Else, start again from the first step.
5. Open the auto mode on the app and make the robot stand up, the robot will navigate along the route computed by the global planning, while using local planning to avoid dynamic obstacles, until it successfully arrives at the destination.

[Caution] To avoid the robot body being classified as an obstacle, only items that exceed a certain height will be identified as obstacles.

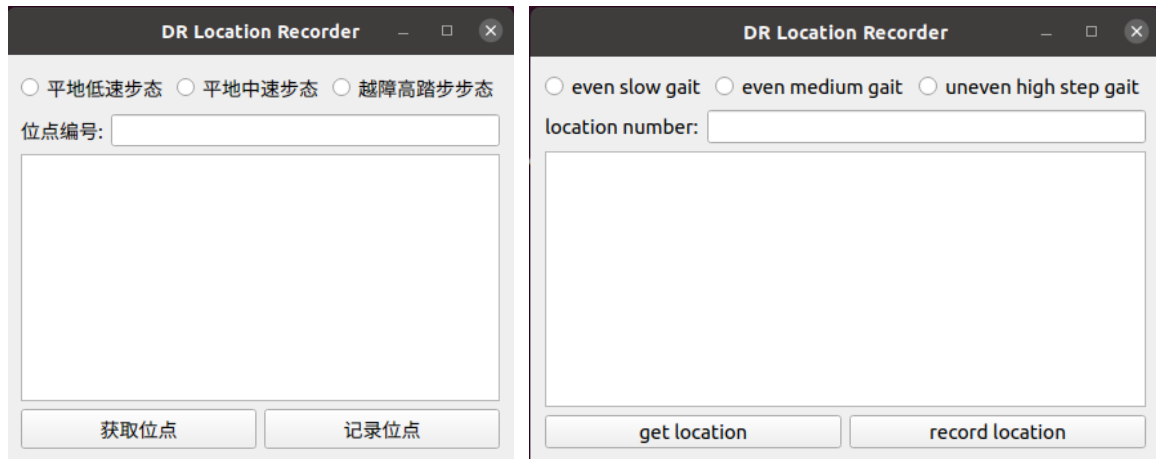
6.3.2 Usage of Multi-point Navigation

This case also provides the function to make the robot autonomously arrive at a series of waypoints in order.

[Caution] Before recording a new route, please check if there are any waypoint files saved before in the `/home/ysc/lite_cog/pipeline/src/pipeline/data` folder, and move them to other folders, to avoid the overwriting.

1. Refer to steps 1 to 3 in 6.2.1 to start the navigation program and initialize the localization, then open a terminal and run the following command to start the pipeline which used for recording a route (consists of many waypoints in sequence):

```
1 cd /home/ysc/lite_cog/pipeline/src/pipeline_tracking/tools
2 python3 location_record.py      # for opening a Chinese interface
3 python3 location_record_en.py  # for opening a English interface
```



2. First the robot should be controlled to arrive at the first waypoint and stand still.

After the point cloud shown in the RViz stops moving, input 1 in the textbox of [location number]. Then click [get location] and the location and orientation information of the robot will be printed. Then click [record location] and a record file named 1.json will appear in /home/ysc/lite_cog/pipeline/src/pipeline/data. Then remote control the robot to the next waypoint, repeat the above operation until all the waypoints are recorded, and then close the window. If it is closed accidentally during recording, just open it again (referring to the first step).

3. Open a terminal, run the following command, and turn on auto mode on the app.

The robot will go to the nearest waypoint and navigate in a loop according to the location number.

```
1 | cd /home/ysc/lite_cog/pipeline
2 | source devel/setup.bash
3 | cd /home/ysc/lite_cog/pipeline/src/pipeline_tracking/scripts
4 | python3 Task.py
```

4. Once the previous operation is finished, you can simply start the navigation program and initialize the localization referring to 6.2.1, and execute step 3 to make the robot follow the previously recorded waypoints for circular navigation when using it again.

6.4 Development

The source code of this case is in `/home/ysc/lite_cog/slam` and `/home/ysc/lite_cog/nav`.

6.4.1 Package Structure

```
/home/ysc/lite_cog/nav
├─ build
├─ devel
├─ src
│   ├── CMakeLists.txt
│   ├── fast_gicp
│   ├── hdl_global_localization
│   ├── hdl_localization
│   ├── navigation
│   └─ ndt_omp
```

1. *hdl_localization* is used for robot localization during navigation.
2. *navigation* is used for path planning.

Users can find the launch files and config files in the corresponding package to develop according to different requirements.

6.4.2 Parameters of Mapping

According to different environment, you can open *mapping_rslidar_indoor.launch* in `home/ysc/jueying_mapping_localization_ws/src/hdl_graph_slam/launch` and modify the following parameters:

```
1 | <param name="distance_far_thresh" value="100.0" />
```

1. When mapping in a large outdoor area, it is advisable to adjust the parameter to 100.
2. When mapping indoors or in an outdoor environment that is not open air, it is recommended to set the parameter to 50.

6.4.3 Parameters of Navigation and Obstacle Avoidance

In `/home/ysc/lite_cog/nav/src/navigation/config` directory, there are five parameter configuration files(.yaml). The most important are the following parameters in

teb_local_planner_params_lite.yaml:

```

1  # Obstacles
2  min_obstacle_dist: 0.20           #Minimum distance from obstacles
3  inflation_dist: 0.4               #Barrier collision buffer size
4  # Robot Omnidirectional velocity & acceleration parameter configuration
5  max_vel_x: 0.7                    #Forward speed limit
6  max_vel_x_backwards: 0.7          #Backward speed limit
7  max_vel_y: 0.4                    #Lateral speed limit
8  max_vel_theta: 0.65               #Rotation angular speed limit
9  acc_lim_x: 0.2                    #Forward and backward acceleration limit
10 acc_lim_y: 0.3                    #Lateral acceleration limit
11 acc_lim_theta: 0.65               #Rotation angular acceleration limit
12 use_proportional_saturation: true
13 # GoalTolerance
14 yaw_goal_tolerance: 0.075          #The larger the parameter
15 xy_goal_tolerance: 0.2             #The higher the directional error of the navigation
16 arrival point
17 free_goal_vel: false

```

And the following parameters in *global_planner_params.yaml*:

```

1  neutral_cost: 75                   #The larger the parameter, the closer to the corner
2  during global route planning

```